Slide 1:
"Welcome, everyone. Today, I will be discussing **Parallel Computing** and its application in handling large datasets. This presentation is brought to you by **Group 7**. Let's dive into how we can apply parallel computing to manage and analyze large datasets efficiently."

Slide 2:
"Our team members are **Baje, Caladiao, Tenio, Ortega, and I am, Gumawid**. Together, we've worked on exploring the benefits and challenges of parallel computing in database management."

Slide 3:
"Our agenda for this presentation are as follow:

1. **First,** we'll discuss the dataset we used in **Dataset Information**.
2. **Secondly,** how we split data for better management by exploring **Partitioning Strategies**.
3. **Third,** how **indexing** improves query performance.
4. **Next, is Benefits and Challenges of Connection Pooling** by managing database connections efficiently.
5. **Lastly, Optimization Recommendations**, which includes best practices for improving performance."

Slide 4:
"Proceeding to our first agenda, Dataset Information. The dataset we selected is a **crime dataset** sourced from **Kaggle**, with a size of **256MB** and over **1 million rows**. It contains detailed records of crimes reported across various regions from **2020 to the present**. Provides valuable insights into crime trends, patterns, and changes in crime rates over time. For our database, **w**e used **DuckDB**, an in-memory database, to manage this dataset."

Slide 5:
"Now, let's talk about our second agenda, **partitioning strategies**. Imagine you have a big box of mixed LEGO bricks where red, blue, green, and yellow pieces all jumbled together. To find a specific color quickly, you'd sort them into smaller boxes by color. Similarly, in databases, this is called **partitioning where it** splits large datasets into smaller, organized groups based on rules like date, category, or user ID."

Slide 6:
What is partionining? For the definition, partitioning is the process of **splitting a large set of data into smaller**, easier-to-manage parts called (partitions), just like sorting LEGO bricks into different boxes.

Bakit natin siya ginagawa? Because when data is split up, it's **easier and faster to search and retrieve information** instead of looking through everything at once.

For example, if you have a list of 20,000 customers and you divide them based on the city they live in, finding people from a specific city will be much quicker.

Slide 7:
" There are numerous partitioning strategies but for our team we utilized **Range Partitioning**. Why? Because it offers:

1. **Efficient Filtering** – Range Partitioning makes it easier to filter data within specific ranges.
2. **Reduced Scanning** – It reduces the amount of data that needs to be scanned for queries.
3. **Parallel Processing** – It enables faster query execution by processing partitions in parallel.

Slide 8:
"**How does partitioning improve query speed in DuckDB?**

1. **Partition Pruning** – Here, only relevant partitions are scanned.
2. **Parallel Processing** – The multiple partitions are processed simultaneously.
3. **Reduced I/O Overhead** – Makes it so that less data is read from disk.
4. **Better Cache Utilization** – It Improves use of memory cache."

Slide 9:
To get started, we downloaded the **crime dataset** from **Kaggle**.

Slide 10:
"Here's how we downloaded the dataset using the **Kaggle API**. We used Python to automate the download process, ensuring we had the latest version of the dataset."

Slide 11:
"If you're new to Kaggle, you'll need to **sign up** or **log in** to download datasets. Once registered, you will be able to download the dataset."

Slide 12:
"To set up our environment, we created a **virtual environment** in Python. This helps isolate dependencies

and ensures our project runs smoothly. We then activated the virtual environment. If you saw the following texts appear on your terminal, it signifies that you activated the virtual environment. Lastly, we installed necessary Python packages like **pandas** and **DuckDB**."

Slide 13:

"We used **pandas** for data handling and manipulation, and **DuckDB** as our in-memory database. Unlike **SQLite** or **PostgreSQL**, DuckDB supports parallel and concurrent queries natively, making it ideal for our activity."

Slide 14:

"In our **operations.py** file, we imported **DuckDB** and **pandas,** optionally you could import type annotations from the typing module. Next step, we also created a class and a dunder method, called **DuckDBOperations** to handle database operations. This class initializes the DuckDB connection and loads the CSV file into the database."

Slide 15:

"Step 3 is to Create a method for loading the csv file to the database. The function **load_csv** method in our class loads the CSV file into DuckDB with a manually defined schema.

Slide 16:

"We also implemented a **get_columns** method to track schema changes and maintain data quality. This method retrieves column information, including statistics like min, max, and distinct values, which are useful for analytics and data governance."

Slide 17:

"To execute queries, we created two functions:

1. **query** – Function for executing SQL commands in a single thread
2. **parallel_query** – function for executing multithreaded queries as DuckDB does not support multiprocessing write operations yet

---

Slide 18:

It is good practice to always create a dunder method, in our case, we implemented it as "_ _ del _ _" or create a context manager for closing a connection with a database using a user-defined class. This is good practice to avoid resource leaks.

Slide 19:
"In **main.py**, we imported the **DuckDBOperations** class and the time module for benchmarking. We also defined a schema for the crime dataset as “crime_schema”. Defining the schema manually ensures we control memory usage and avoid issues with inferred data types."

Slide 20:
We initialized the **DuckDBOperations** class to create a connection to the database and load the CSV file that will be used. At step 11, we then performed benchmarking queries without partitioning to compare single-threaded and parallel execution times.

Slide 21:
"Next, implement a **range partitioning** strategy in our SQL queries. As you can see on the screen , we partitioned the data based on the **Vict_Age** column, dividing it into ranges like 0-18, 19-35, 36-60, and 60+. This allowed us to perform faster queries on specific age groups." After that, perform the bench marking again.

Slide 22:
"After running the queries, close the connection by using “del db”. Then we executed the script. As you can see at the time difference, the results showed that **parallel execution** was significantly faster than single-threaded execution, especially with large datasets."

Slide 23:
"**Findings:**

1. **Single-threaded vs. Parallel Execution** –Parallel execution is faster, especially with large datasets but can become hardware intensive quickly depending on the size of a dataset.
2. **Partitioning Effect** –Depending on data distribution and partitioning strategy, improper application of the partitioning could lead to data loss due to uneven distribution with each partition or could break the partition.

Slide 24:
"For our next agenda, let’s talk about **indexing**. Indexing stores the values of one or more columns from the table and points to where those values are located in the actual data.

What is indexing in a database? Indexing in a database creates a data structure (it’s like a table of contents) that helps the database quickly find specific rows or values in a large dataset. This helps the database quickly find specific rows or values."

Slide 25:
"**How does indexing enhance performance?**

1. **Faster Searches** – Quickly locate specific rows.
2. **Faster Sorting and Filtering** – It efficiently sort and filter data.
3. **Optimized Join Operations** – Improves performance of join queries.
4. **Improved Query Performance** – It speeds up SELECT queries."

Slide 26:
"Next subtopic is the **Impact of Indexing on Performance:**

1. **Improves Query Speed** – Especially for search, sort, and group operations.
2. **Drawbacks or Costs of Indexing** – Slower writes, increased disk space usage, and added complexity.

When should we Use Indexing? Indexes should be used for columns frequently queried, but over-indexing should be avoided."

Slide 27:
"Continuing with our steps, inside the activity_1 folder, we created a new **benchmarks.py** file to test the performance of queries with indexing. Inside it, the script imports necessary packages for benchmarking and defines the schema for the crime dataset."

Slide 28:
"In **benchmarks.py**, we defined the schema for the crime dataset then, we initialized the **DuckDBOperations** class. We then loaded the CSV file and schema into the database."

Slide 29:
"We created an index on the **Vict_Age** column, which is “use CREATE INDEX idx_vict_age ON crime_data(Vict_Age). We do this to improve query performance. We then executed queries with indexing in a single-thread, and another with indexing and range partitioning.

Slide 30:
Next, execute the previous query command in parallel and then close the connection after the benchmarking was complete. After executing the script, the result showed that indexing significantly improved query performance, especially for range-based filters.

Slide 31:
"Now, let us discuss our **Findings:**

1. **Performance Improvement with Indexing** – Queries executed faster after indexing.
2. **Range Partitioning** – Combined with indexing, it further optimized searches.
3. **Parallel Execution** – DuckDB's parallel execution across threads enabled the fastest query time.

Slide 32:
"Let's turn our attention now to our next agenda, which is **connection pooling**. Connection pooling offers key benefits:

1. Reduced Overhead - Reusing existing database connections saves time compared to constantly creating new ones.
2. Concurrency Management – Prevents overloading of databases when many users access it at once
3. Scalability – It simplifies handling increased traffic and scaling your system.
4. Improved Latency - Applications get data from the database faster.

Moving on to its disadvantages:

1. **Resource contention** can occur when many requests compete for a limited number of connections.
2. **Idle connections**, though readily available, can still consume resources.
3. Finally, keeping the pool organized and running smoothly takes up resources.

Slide 33:
"However, **connection pooling can lead to deadlocks** if not managed properly. Deadlocks occur when multiple connections wait for each other to release resources, causing a cyclic dependency where no process can proceed."

How Connection Pooling Can Lead to Deadlocks

1. Connection Starvation - This can happen if too many requests need connections and the pool runs out

2. Incorrect Transaction Handling - Improper handling of database transactions, like holding onto connections for too long.

3. Resource Contention - If many things are trying to use the same limited connections.

4. Exhaustion of Connections - if connections aren't managed carefully, pooling can create a traffic jam where everything grinds to a halt.

5. Nested Queries or Dependency Chains – involves complex situations where one query depends on another

6. Improper Pool Configuration - pool isn't set up correctly, risk of deadlock are more likely.

Slide 34:

Deadlocks can slow down databases, but we can prevent them by following best practices: such as **setting timeouts**, **releasing connections efficiently**, **choosing the right isolation levels**, **keeping transactions short**, **monitoring pool usage**, and **increasing pool size**, these steps help ensure smooth database operations."

Does Deadlock Risk Depend on the Database Type? Yes. **PostgreSQL** and **MySQL** have built-in deadlock detection, **SQLite** avoids deadlocks by using a single connection with exclusive locking, and **DuckDB**, does not support deadlock detection since it's optimized for analytical workloads rather than transactional concurrency

Slide 35:
"**Does DuckDB support connection pooling?** No, to summarize this, DuckDB does not natively support connection pooling. However, its lightweight connections reduce the need for pooling in many cases."

Slide 36:
To give an example of databases that support connection pooling natively are MySQL, MariaDB, and Microsoft SQL Server.

"To simulate connection pooling, we used **thread pooling**. This allowed us to manage concurrent queries efficiently, even though DuckDB doesn't natively support connection pooling."

Slide 37:
Is thread pooling the same as connection pooling? The answer is no. They are two distinct concepts with different focuses,but they share a common objective, which is resource optimization.

("**Thread Pooling vs. Connection Pooling:)** To make this comparison short, **Thread Pooling** manages concurrent tasks using a fixed number of threads. While, **Connection Pooling** manages database connections to reduce overhead.

Also, thread pooling mimics connection pooling but it lacks the features like persistent connections and idle connection management."

Slide 38:
To sum up this slide, thread pooling allowed us to evaluate the performance gains of concurrent query execution. DuckDB connections are lightweight and in-memory, reducing the need for actual pooling. In

case a project requires actual connection pooling, transitioning to databases that support it like MySQL would be better.

Slide 39:
Continuing on to our steps, "Inside the activity_1 folde, we created a **pooling.py** file to benchmark connection pooling. Inside the this file, import the necessary packages because this defines the schema for the dataset."

Slide 40:
"In here, we defined the schema and made the database persistent instead of in-memory. Additionally, we have defined the path for the CSV file and the table name."

Slide 41:
At step 31, initialized the **DuckDBOperations** class and load the CSV file into the database. We then defined a function to retrieve connections from the pool which is called “get_connection”.

Slide 42:
"Next step, it would be creating functions for queries with **partitioning** as “partitioned_query” and **indexing** as “indexing_query”. These functions used connection pooling to improve query performance."

Slide 43:
Create another function and it will be use for queries with both **indexing** and **partitioning**, then another function for a query with indexing, partitioning, and execute in parallel with connection pooling. This will combine the benefits of both strategies.

Slide 44:
After those functions, we will benchmark all scenarios. We initialized DuckDBOperations and load the csv file. The connection will automatically close once the context manager is finished.

Slide 45:
Execute the script in order to benchmark the scenarios.

Our Findings are as follows:

1. **Partitioning with Connection Pooling** – It improved performance by dividing data into chunks.
2. **Indexing with Connection Pooling** – Faster lookup performance for indexed columns.

3. **Lastly, Parallel Execution** – It achieved the best performance by distributing workload across threads. This is ideal for high-traffic environments but may not be necessary for low workloads.

Slide 46:
For the next agenda, we'll talk about the "**Summary of Benefits:**

1. **Partitioning** – Improves manageability and query speed.
2. **Indexing** – Enhances reading, aggregating, and searching datasets.
3. **Parallel Execution** – Speeds up query processing by splitting the workload across multiple threads.
4. **Connection Pooling** – Reduces connection overhead.

Slide 47:
Summary of Trade-offs:
This activity, implementing both parallel execution and connection pooling can be overkill for small datasets or low-traffic applications. These strategies are best suited for high-traffic applications that access database frequently.

Slide 48:
And the last agenda, is our **Recommendations:**

1. First is to use **Range Partitioning** for tables with millions of values, especially those with time-related fields. This technique divides the table into smaller partitions based on a continuous range of values within a specific or certain range.
2. Second is to utilize **Indexing** because it significantly enhances database performance. It enables faster searches, facilitates efficient sorting, and optimizes join operations, leading to quicker query execution.

Slide 49:
"**Recommendations (continued):**

3. Third, **Parallel execution** is a powerful technique for accelerating data processing. It's particularly beneficial for large datasets, significantly speeding up table scans and enabling high-performance analytics and machine learning applications.
4. Lastly, **Connection pooling** is essential for efficient database management. By reusing existing connections, it reduces overhead, improves concurrency, and ensures faster response times for multiple queries, especially in high-traffic environments.

Slide 50:
To summarize our presentation, we used a crime dataset in DuckDB. We implemented range partitioning to split data into smaller and manageable parts, indexing for faster queries, and thread pooling as an alternative to connection pooling, while leveraging parallel query execution for concurrent processing. That's all for us. Thank you for listening.

Slide 51:
"**References:**
We've included a list of references used in this presentation. These resources provide further reading on partitioning, indexing, connection pooling, and parallel computing."